SISTEMI E RETI · CLASSE QUINTA

Il livello applicazione

Modelli client-server e peer-to-peer, Telnet, FTP, HTTP e HTTPS, posta elettronica (SMTP, POP3, IMAP), DNS

Come leggere questi appunti. I riquadri blu contengono le **definizioni** da sapere, quelli verdi gli **esempi**, quelli gialli i punti in cui è **facile sbagliare**. Le parti prese dalle slide del docente, che negli appunti non c'erano, sono segnalate sotto il titolo del paragrafo. In fondo trovi uno schema riassuntivo con le porte da ricordare, le domande di autoverifica con le risposte e l'elenco di ciò che è stato corretto rispetto agli appunti.

1. Il livello applicazione e i suoi modelli

Il **livello applicazione** è il livello più alto della pila di protocolli: è quello con cui hanno a che fare direttamente i programmi che usiamo, come il browser, il client di posta o un programma per trasferire file. I livelli sotto (trasporto, rete, ecc.) si occupano di far arrivare i dati; il livello applicazione stabilisce *che cosa* si dicono i due programmi e *in che modo*.

1.1 Che cosa definisce un protocollo applicativo

Questa parte non è negli appunti: viene dalle slide (sl. 3).

Ogni protocollo di livello applicazione è un insieme di regole che fissa quattro cose:

- i **tipi di messaggi** che si scambiano (per esempio richiesta e risposta);
- la **sintassi** di ciascun messaggio, cioè come è scritto e da quali campi è composto;
- la **semantica** dei campi, cioè che cosa significa ciascuno;
- le **regole del dialogo**: chi parla per primo, quando si risponde, quando si chiude.

1.2 Il modello client-server

I dettagli sul server sempre in attesa vengono dalle slide (sl. 4).

A livello applicazione si usano due modelli di comunicazione. Il più diffuso è il **client-server**: c'è un programma, il **server**, che offre un servizio, e uno o più programmi, i **client**, che lo richiedono. Il server è sempre acceso e in attesa di richieste, per questo ha un **indirizzo IP statico** (fisso): i client devono sapere sempre dove trovarlo. Il client invece si collega solo quando gli serve qualcosa.

Usano questo modello il **web** (www), la **posta elettronica** e il trasferimento di file.

1.3 Il modello peer-to-peer

Login server, tracker, peer e gli aspetti delle applicazioni P2P vengono dalle slide (sl. 5-7).

Nel modello **peer-to-peer** (P2P, «da pari a pari») non c'è distinzione fissa tra client e server: ogni computer, detto **peer**, può sia chiedere sia offrire risorse. Sono P2P le applicazioni di file sharing come BitTorrent e quelle di videochiamata e telefonia via Internet, come Skype e Teams. Un'applicazione P2P, comunque, può anche funzionare in parte come client-server.

In una rete P2P tipica si trovano tre elementi:

- **login server**: un portale web a cui il peer si collega per verificare se i servizi che cerca sono disponibili;
- **tracker**: fornisce al nuovo utente l'elenco dei peer disponibili, cioè di chi ha la risorsa;
- **computer peer**: macchine anonime che si collegano quando ne hanno bisogno.

Le applicazioni P2P devono quindi risolvere tre problemi: la **ricerca** della risorsa, la sua **locazione** (su quali peer si trova) e il **trasferimento dei dati**, che può avvenire in modalità **push** (è chi ha il dato a inviarlo) o **pull** (è chi lo vuole a richiederlo).

	Client-server	Peer-to-peer
Ruoli	fissi: il server offre, il client chiede	ogni peer può chiedere e offrire
Disponibilità	server sempre acceso, IP statico	i peer si collegano quando serve
Esempi	web, posta elettronica, trasferimento file	BitTorrent, Skype, Teams

2. Telnet

2.1 Come funziona

Telnet è un protocollo client-server che permette di usare un computer remoto da riga di comando, come se fossimo seduti davanti a lui: il client è un'applicazione testuale, il server gira sull'host remoto. Fa quindi lo stesso lavoro di **SSH**, ma con una differenza decisiva: Telnet è basato su **TCP** e la comunicazione **non è criptata**. Tutto quello che si scrive, password compresa, viaggia in chiaro sulla rete.

Di norma il server Telnet è in ascolto sulla **porta 23**. Su Windows non si usa, perché il client Telnet è **disabilitato di default** (va attivato a mano).

La sintassi del comando sul client è:

```
telnet [opzioni] [host remoto] [porta]
```

Se la porta non si scrive, si usa la 23. L'host remoto si può indicare con il nome o con l'indirizzo IP.

> **ATTENZIONE**
> Telnet e SSH servono alla stessa cosa (accesso remoto da terminale), ma non sono equivalenti: **SSH cifra** la comunicazione, Telnet no. Per questo SSH è nato come sostituto sicuro di Telnet, che oggi si usa quasi solo in reti di test o di laboratorio.

2.2 I comandi principali

Questa parte non è negli appunti: viene dalle slide (sl. 10).

Una volta avviato il client, la sessione si gestisce con pochi comandi:

Comando	A cosa serve
open host [porta]	apre la sessione Telnet verso l'host
close	chiude la sessione
display	mostra i parametri della sessione
send code	invia caratteri speciali al server
status	mostra lo stato attuale della sessione

La sicurezza si limita a username e password, che però, come tutto il resto, viaggiano in chiaro. Un programma molto usato come client Telnet (e SSH) è **PuTTY**.

3. FTP (File Transfer Protocol)

3.1 Due sessioni e due porte

FTP serve a **trasferire file**. Come struttura è client-server, anche se può lavorare pure in modo peer-to-peer. Di solito lo si usa tramite applicazioni come **FileZilla**: il client ha in genere un **IP privato** (è un PC in una rete domestica o aziendale), il server un **IP pubblico** raggiungibile da Internet.

Una sessione FTP è in realtà fatta di **due sessioni**: nella prima client e server stabiliscono i meccanismi della comunicazione (chi è l'utente, verifica della password); nella seconda avviene l'effettiva trasmissione dei file. Per tenere separati i due compiti, FTP usa **due canali**, cioè due porte:

- **porta 21**, canale comandi: qui passano i comandi del client (del tipo «esegui», «leggi», «scarica questo file») e le risposte del server;
- **porta 20**, canale dati: qui passa il file vero e proprio.

Per collegarsi bisogna **autenticarsi** con nome utente e password, ma **senza crittografia**: come in Telnet, le credenziali viaggiano in chiaro.

> **DEFINIZIONE**
> **FTP** è il protocollo per il trasferimento di file tra client e server; usa la **porta 21** per i comandi e la **porta 20** per i dati, e non cifra né le credenziali né i file.

3.2 Modalità active e passive

Questa parte non è negli appunti: viene dalle slide (sl. 13-15).

Il canale comandi lo apre sempre il client. Il canale dati, invece, può essere aperto in due modi diversi, e qui nascono le due modalità di FTP.

In **FTP active** il client non si collega alla porta dati del server: gli dice soltanto su quale porta è in ascolto (comando PORT), ed è il **server** che apre il canale dati, dalla sua porta 20 verso quella porta del client. Questo crea problemi di sicurezza **lato client**: il client deve accettare una connessione in

arrivo dall'esterno, cosa che firewall e reti con IP privato di solito bloccano, e tenere porte aperte è comunque un rischio.

In **FTP passive** il client chiede la modalità passiva (comando PASV); il server apre una porta casuale per i dati e ne comunica il numero; è il **client** ad aprire il canale dati verso quella porta. Così tutte le connessioni partono dal client, e il problema sparisce.

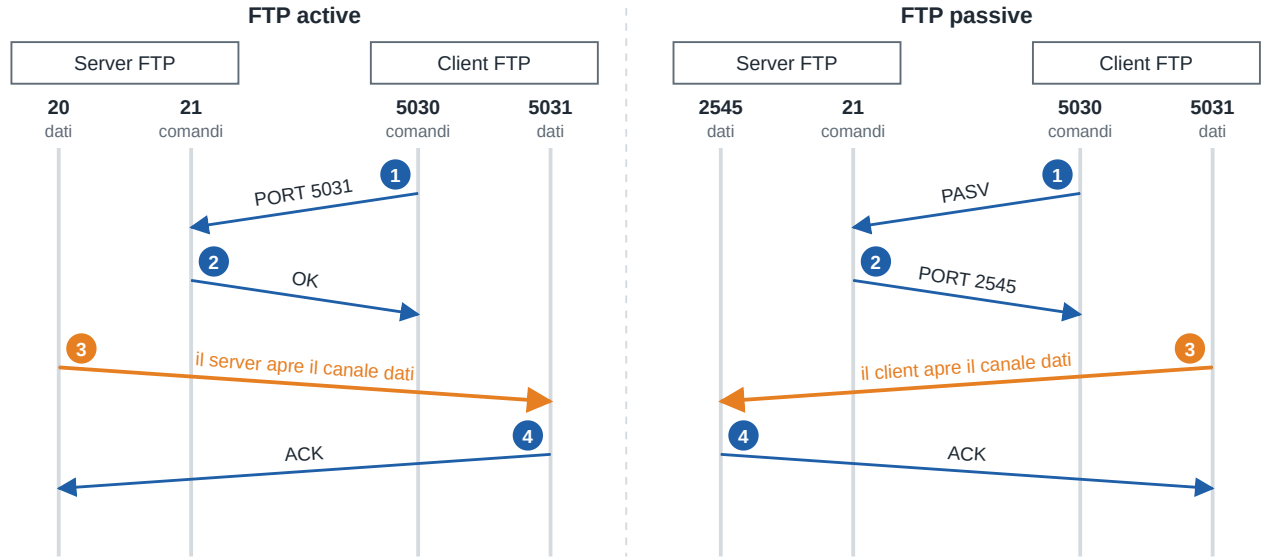


Figura 1 – FTP active e FTP passive. In arancio il passo che cambia: in active il canale dati lo apre il server, in passive il client.

> **ATTENZIONE**
> Active e passive si riferiscono al **server**: in active è il server che apre «attivamente» la connessione dati; in passive il server resta «passivo» e aspetta che sia il client a collegarsi.

3.3 Accesso al server e vulnerabilità

Questa parte non è negli appunti: viene dalle slide (sl. 16).

Al server FTP si accede in due modi:

- **accesso utente**: servono nome utente e password;
- **accesso anonimo**: si entra con il nome utente **anonymous**; è molto usato per distribuire dati pubblici.

Il punto debole di FTP è sempre lo stesso: **nessuna cifratura**. Password e dati viaggiano in chiaro e chi intercetta il traffico può leggerli.

3.4 FTP sicuro con TLS

Per rendere FTP sicuro si aggiunge **TLS** (**Transport Layer Security**), un livello che si mette tra il trasporto (TCP) e l'applicazione (FTP) e si occupa della crittografia. FTP continua a funzionare come prima, ma tutto quello che passa viene cifrato. Il risultato si chiama **FTPS** (FTP over TLS).

> **ATTENZIONE**
> **FTPS** e **SFTP** non sono la stessa cosa. FTPS è il normale FTP con l'aggiunta di TLS. SFTP (SSH File Transfer Protocol) è un protocollo diverso, che trasferisce file sfruttando SSH.

4. HTTP e HTTPS

4.1 Browser e web server

HTTP (**HyperText Transfer Protocol**) è il protocollo del web ed è client-server. Il client è il **browser** (Chrome, Edge, Firefox…), il server è il **web server**, cioè il programma che contiene le pagine del sito: esempi sono **Apache** e **IIS** (Internet Information Services) di Microsoft.

Il dialogo è semplice: il client invia una richiesta per una pagina web, detta **HTTP request**, e il server risponde con una **HTTP response** che contiene la pagina, di solito scritta in HTML.

4.2 Una connessione HTTP passo per passo

Già dalla prima versione, **HTTP/1.0**, i dati trasportati sono in genere pagine **HTML** o **XML**, e ogni connessione segue sempre gli stessi quattro passi:

1. **apertura**: il browser apre una connessione TCP verso la **porta 80** del server;
2. **richiesta HTTP**: il browser chiede la risorsa;
3. **risposta HTTP**: il server invia la risorsa;
4. **chiusura**: arrivata la risposta, la connessione viene chiusa.

Questa struttura è rimasta sostanzialmente la stessa ancora oggi. Ne derivano tre caratteristiche di HTTP:

- **testuale**: i messaggi sono scritti in caratteri ASCII (anche se i dati inviati dal server possono essere immagini, suoni, ecc.);
- **pseudoanonimo**: il server conosce solo l'indirizzo IP del client, quindi considera due richieste dallo stesso IP come provenienti dallo stesso utente;
- **stateless** («senza stato»): non esiste una sessione; tra due richieste non c'è nessuna relazione, anche se riguardano la stessa pagina.

> **DEFINIZIONE**
> Un protocollo è **stateless** quando il server non conserva memoria delle richieste precedenti: ogni richiesta è trattata come se fosse la prima.

4.3 Le versioni di HTTP

Le novità di HTTP/1.1 vengono dalle slide (sl. 22).

HTTP si è evoluto per diventare più veloce e più sicuro. Il limite principale di HTTP/1.0 è proprio la chiusura dopo ogni risposta: una pagina con dieci immagini richiede dieci connessioni. **HTTP/1.1** introduce le **connessioni persistenti**, che restano aperte per più richieste; inoltre browser e server

possono **negoziare il formato** dei dati, e i nodi intermedi (i **proxy**, vedi 4.8) possono tenere una **cache**, cioè una copia locale dei dati, per non andarli a riprendere ogni volta dal server.

Versione	Che cosa cambia
HTTP/1.0	richiesta e risposta su TCP; la connessione si chiude dopo ogni risposta
HTTP/1.1	connessioni persistenti, negoziazione del formato, gestione della cache
HTTP/2	nei browser si usa solo insieme a **TLS**, quindi su connessioni HTTPS
HTTP/3	usa **UDP** al posto di TCP; è supportato dai principali browser

4.4 HTTPS

La porta 443 e la regola sui contenuti vengono dalle slide (sl. 34).

HTTPS è HTTP reso sicuro: come per FTP, tra TCP e HTTP si inserisce il livello **TLS**, che cifra e autentica la comunicazione e impedisce di intercettarne i contenuti. Nato per le transazioni commerciali, oggi è usato quasi ovunque. Rispetto a HTTP cambiano due cose: si usa la **porta 443** invece della 80, e si usa il protocollo **TLS** (in passato SSL).

Se una pagina è in HTTPS, anche tutti i contenuti che carica devono esserlo: altrimenti una parte viaggerebbe in chiaro e la protezione non avrebbe senso.

> **ATTENZIONE**
> HTTPS non è una versione di HTTP come HTTP/2 o HTTP/3: è **HTTP (di qualunque versione) trasportato su TLS**. Esisteva già con HTTP/1.1; con HTTP/2 i browser hanno solo deciso di usarlo esclusivamente su connessioni cifrate.

4.5 L'URL

La struttura completa dell'URL, con porta e fragment, viene dalle slide (sl. 24-27).

Ogni risorsa del web ha un indirizzo simbolico, l'**URL** (**Uniform Resource Locator**): è quello che si scrive nella barra del browser e a cui puntano i link. La sua forma completa è:

```
protocollo://[username[:password]@]host[:porta][/percorso][?querystring][#fragment]
```

Le parti tra parentesi quadre sono facoltative.

- **protocollo**: dice al browser quale protocollo usare, di solito **http** o **https**. Il browser lo mostra anche con un simbolo: un lucchetto (o scudo) per HTTPS, un segnale di pericolo per HTTP non protetto;
- **username e password**: un tempo si potevano scrivere nell'URL per fare il login, e venivano inviate con la richiesta GET. Oggi è considerato **pericoloso**, perché si presta al phishing: alcuni browser hanno eliminato la funzione, altri avvisano l'utente;
- **host**: il server su cui si trova la risorsa, scritto come nome di dominio (es. esempio.com) o come indirizzo IP. Il nome viene tradotto in IP dal **DNS** (sezione 6);
- **porta**: facoltativa, si scrive solo se non è quella standard (80 per HTTP, 443 per HTTPS);
- **percorso** (path): tutto quello che segue il dominio; ogni / separa una cartella sul server. Se manca, il server invia la pagina di default;

- **query string**: dopo il «?», passa al server dei parametri, per esempio quello che si sta cercando;
- **fragment**: dopo il «#», indica un punto preciso dentro la pagina. Serve solo al browser e **non viene inviato al server**.

> **ESEMPIO · SCOMPORRE UN URL**
> In **https://www.esempio.com:443/news/sport/?q=calcio#risultati**: «https» è il protocollo; «www.esempio.com» è l'host, dove «www» è il nome del web server ed «esempio.com» il dominio del sito; «443» è la porta; «/news/sport/» è il percorso (cartella news, sottocartella sport); «q=calcio» è la query string con la parola cercata; «risultati» è il fragment, il punto della pagina su cui il browser si posiziona.

4.6 Stateless e cookie

Questa parte non è negli appunti: viene dalle slide (sl. 28-29).

HTTP è stateless per una scelta precisa: senza dover ricordare nulla delle connessioni, il server può gestire richieste rapide e l'utente può saltare velocemente da un web server all'altro seguendo i link. Il prezzo è che l'interazione con l'utente diventa scomoda: se ci si autentica su una pagina, passando a un'altra pagina il server non si ricorda più di noi e dovremmo reinserire le credenziali.

Per risolvere il problema sono nati i **cookie**: piccoli file di testo che il web server invia al browser e che il browser rimanda indietro, senza modificarli, a ogni richiesta successiva. In questo modo il server «riconosce» il client. I cookie servono a:

- implementare l'**autenticazione**, per esempio restare loggati;
- **memorizzare dati** utili alla sessione di navigazione (come il carrello di un negozio online);
- **tracciare la navigazione** dell'utente, anche a fini pubblicitari.

4.7 I metodi e i messaggi HTTP

Le descrizioni di OPTIONS e TRACE, la struttura dei messaggi e i codici di stato vengono dalle slide (sl. 31-32).

Nella richiesta, il client indica con un **metodo** che cosa vuole fare sulla risorsa:

Metodo	Che cosa fa
GET	richiede una risorsa al server (pagina HTML, immagine…)
HEAD	richiede solo l'header, senza la risorsa (utile per la diagnostica)
POST	invia informazioni al server (per esempio i dati di un modulo)
PUT	richiede l'upload di un file sul server
DELETE	richiede la cancellazione di un file dal server
OPTIONS	richiede l'elenco dei metodi che il server accetta
TRACE	traccia la richiesta, mostrando come viene trattata dal server
CONNECT	crea una connessione attraverso un proxy

> **ATTENZIONE**
> GET e HEAD si somigliano: entrambi chiedono una risorsa, ma con HEAD il server risponde **solo con l'header**, senza il contenuto. POST e PUT si somigliano anche loro: POST **invia dati** al server, PUT chiede di **caricare un file**.

I messaggi HTTP sono di due tipi, entrambi formati da tre parti.

La **request** (dal client al server) contiene una **riga di richiesta** con metodo, risorsa richiesta (URI, Uniform Resource Identifier) e versione, poi una sezione di **header** con informazioni aggiuntive e infine il **body**, che contiene gli eventuali dati inviati.

La **response** (dal server al client) contiene una **riga di stato** con un **codice di risposta a 3 cifre**, poi l'**header** (per esempio il tipo di server e il **Content-Type**, cioè il tipo di contenuto restituito) e infine il **body** con il contenuto vero e proprio.

> **ESEMPIO · RIGA DI RICHIESTA**
> **GET /info.html HTTP/1.1** significa: «con il metodo GET, dammi la risorsa /info.html; parlo la versione 1.1 di HTTP».

Codice	Significato
200 OK	richiesta riuscita, la risorsa è nel body
400 Bad Request	la richiesta è scritta male
403 Forbidden	accesso alla risorsa vietato
404 Not Found	la risorsa non esiste
500 Internal Server Error	errore interno del server
505 HTTP Version Not Supported	il server non supporta la versione di HTTP usata

4.8 Il proxy HTTP

Questa parte non è negli appunti: viene dalle slide (sl. 33).

> **DEFINIZIONE**
> Un **proxy** è un programma che si mette tra client e server e fa da tramite: il client invia la richiesta al proxy, il proxy la inoltra al server e poi gira la risposta al client.

Poiché tutte le richieste passano da lui, il proxy può essere usato per:

- **connettività**: un'intera rete privata può accedere a Internet attraverso un unico computer;
- **privacy**: il server vede l'IP del proxy, non quello vero del client;
- **caching**: conserva per un certo tempo le risposte, così le richieste ripetute non arrivano al server;
- **monitoraggio**: tiene traccia delle pagine visitate;
- **amministrazione**: applica regole per decidere quali richieste inoltrare e quali rifiutare.

5. La posta elettronica: SMTP, POP3, IMAP

La posta elettronica usa protocolli diversi per i due momenti del suo percorso: uno per **inviare** i messaggi, altri due, in alternativa tra loro, per **riceverli**.

5.1 SMTP: l'invio

La parte sulla sicurezza e sullo spam viene dalle slide (sl. 39).

SMTP (**Simple Mail Transfer Protocol**) si occupa dell'**invio** della posta: porta il messaggio dal mittente al suo server di posta e poi da un server all'altro fino a quello del destinatario. Usa TCP sulla **porta 25**.

SMTP è nato senza pensare alla sicurezza: non verifica chi è il mittente né se è autorizzato a inviare, quindi **chiunque può spedire un'email a chiunque**; inoltre i messaggi viaggiano **in chiaro**. Anche qui la soluzione è **TLS**: con la cifratura l'invio dal client passa sulla **porta 465** (o **587**), mentre la porta 25 resta in uso per lo scambio tra server di posta.

Proprio perché inviare è così facile, esiste lo **spam**: l'invio in massa di messaggi non richiesti, pubblicitari o contenenti link dannosi, che non si può impedire alla radice.

5.2 POP3 e IMAP: la ricezione

Porte, blocco della casella, cartelle e webmail vengono dalle slide (sl. 43-47).

Per leggere la posta che è arrivata nella propria casella (**mailbox**) sul server si usa uno di due protocolli, che si distinguono per una cosa sola: dove restano i messaggi.

POP3 (**Post Office Protocol**, versione 3) **scarica** le email, allegati compresi, sul client e di norma le **cancella dal server**. All'avvio il client si collega al server e scarica tutto ciò che è arrivato dall'ultima connessione. È comodo per leggere la posta offline, ma una volta scaricati i messaggi non si possono più leggere da un altro dispositivo. Durante la consultazione la casella viene **bloccata**, per evitare accessi contemporanei. Usa la **porta 110** (**995** con TLS); senza TLS anche username e password viaggiano in chiaro.

IMAP (**Internet Message Access Protocol**) invece **lascia tutto sul server**: il client mostra i messaggi e si sincronizza con il server. Per questo si può leggere la stessa posta da più dispositivi, e sono permesse **connessioni simultanee** alla stessa casella. IMAP tiene traccia dello stato di ogni messaggio (letto, risposto) e permette di organizzare le email in **cartelle**. Usa la **porta 143** (**993** con TLS).

	POP3	IMAP
Dove restano i messaggi	scaricati sul client, cancellati dal server	restano sul server
Più dispositivi	no	sì, client e server sincronizzati
Accessi contemporanei	casella bloccata durante la lettura	permessi
Lettura offline	sì	serve la connessione
Porte	110 (995 con TLS)	143 (993 con TLS)

> **ATTENZIONE**
> SMTP serve solo a **inviare**; POP3 e IMAP servono solo a **ricevere**. Un client di posta li usa quindi insieme: SMTP per spedire, POP3 oppure IMAP per leggere.

Esiste infine la **webmail** (Gmail, Outlook.com, Libero…): la posta si legge e si scrive direttamente da un sito web, senza scaricarla sul computer.

6. DNS

Tutta questa sezione non è negli appunti: viene dalle slide (sl. 48-55).

Per raggiungere un server serve il suo indirizzo IP, ma per le persone è molto più facile ricordare un nome come www.youtube.com. Il **DNS** fa da «rubrica» di Internet.

> **DEFINIZIONE**
> Il **DNS** (**Domain Name System**) è il sistema che permette di identificare un server con un **nome** invece che con il suo indirizzo IP, traducendo il nome nell'indirizzo. È un **database distribuito e gerarchico** che segue il modello client-server.

6.1 La gerarchia dei domini

I nomi sono organizzati ad albero. In cima c'è la **radice**; subito sotto ci sono i **domini di primo livello** (.it, .com, .org…). Poi vengono i **domini intermedi**, che contengono a loro volta altri sottodomini, e infine i **domini foglia**, che non hanno sottodomini e contengono solo host.

Un nome si legge quindi **da destra verso sinistra**, dal generale al particolare.

> **ESEMPIO · WWW.IETF.ORG**
> «www» è il nome del web server, che si trova nel dominio «ietf» (l'organizzazione IETF, Internet Engineering Task Force), che è contenuto nel dominio di primo livello «org», che discende dalla radice.

6.2 I resource record

Ogni dominio conserva le sue informazioni in strutture dette **resource record** (RR). Dato il nome di un host, il DNS cerca il resource record che associa quel nome al suo indirizzo IP. Ogni record ha un campo **Type** che dice che tipo di informazione contiene.

Tipo	Che cosa contiene	Esempio
A	l'indirizzo IPv4 dell'host	pingu.di.school.it A 198.45.30.165
AAAA	come A, ma per indirizzi IPv6	
CNAME	un alias: un secondo nome per lo stesso host	www.di.school.it CNAME pingu.di.school.it
MX	i server di posta a cui inviare le email per quel dominio	
NS	il name server responsabile del dominio	di.school.it NS name1.di.school.it

Negli esempi, pingu è una macchina del dipartimento di informatica (di) della scuola school.it.

6.3 La risoluzione dei nomi

Lo spazio dei nomi è diviso in **zone** separate. Ogni zona ha un **name server principale** (DNS primario) e dei **name server secondari** (DNS secondari), che prendono le informazioni dal primario. Quando un'applicazione ha bisogno di un indirizzo, invia un'interrogazione (**query**): il server risponde direttamente, se ha il dato, oppure lo cerca presso altri server.

Questo processo si chiama **risoluzione dei nomi** e può essere di due tipi:

- **iterativa**: al server si chiede la migliore risposta che conosce già, anche se è solo l'indicazione di un altro server a cui rivolgersi;
- **ricorsiva**: al server si chiede di rispondere in modo completo, facendo lui tutte le ricerche necessarie. Normalmente le query sono di questo tipo.

Ecco come viene risolto il nome www.company.com, partendo dal computer dell'utente (il **resolver**):

1. il resolver chiede al suo **default DNS server** l'IP di www.company.com (richiesta di tipo A);
2. se il default DNS server non conosce l'indirizzo, interroga uno dei **root server**;
3. il root server risponde indicando il server responsabile dei nomi .com;
4. il default DNS server rivolge la domanda al server di .com;
5. il server di .com non sa risolvere il nome completo, ma conosce e comunica il server responsabile di company.com;
6. il default DNS server chiede a quest'ultimo;
7. il name server di company.com restituisce l'indirizzo IP (record A);
8. il default DNS server invia l'indirizzo al resolver.

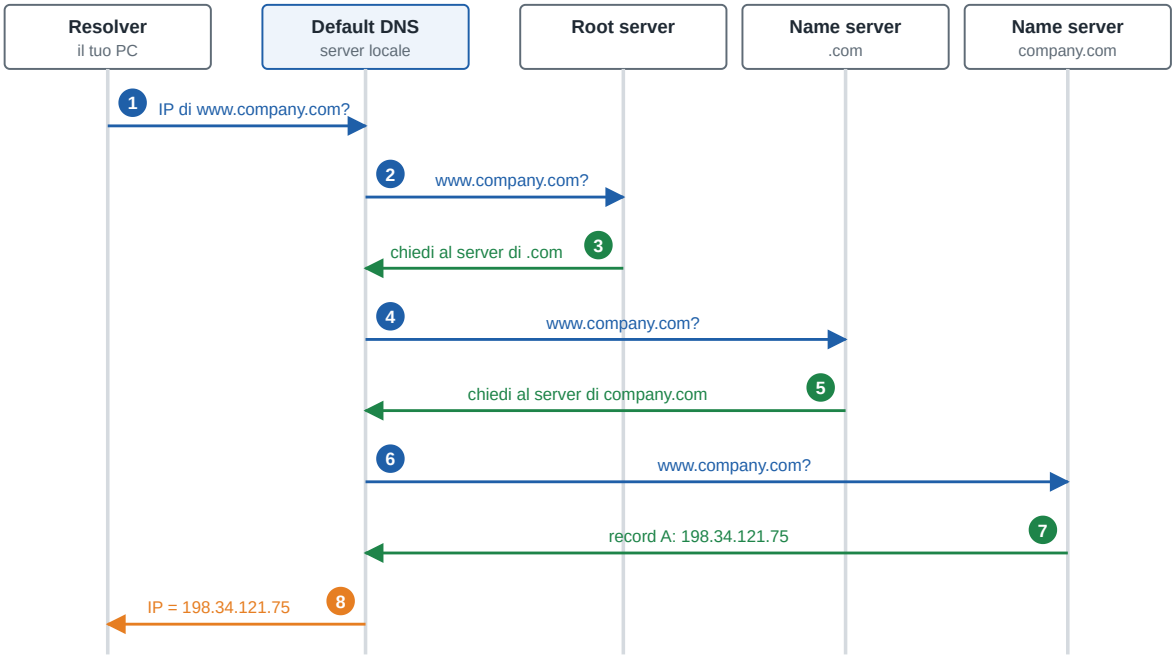


Figura 2 – Risoluzione di www.company.com. Per il resolver la query è ricorsiva (fa una domanda e riceve la risposta completa); il default DNS server invece interroga i server uno dopo l'altro.

6.4 La sicurezza del DNS

Il DNS è un punto molto delicato per la sicurezza, perché:

- **non è autenticato**: la risposta potrebbe arrivare non dal DNS server giusto, ma da un'altra macchina;
- è **lento**, quindi qualcuno può intercettare la richiesta e rispondere al posto del server vero (**spoofing**, cioè «falsificazione»);
- non ha meccanismi per proteggere l'**integrità** delle informazioni.

In questo modo un attaccante può associare a un nome l'indirizzo IP di un **server malevolo**: l'utente scrive l'indirizzo giusto, ma finisce su un sito falso. La soluzione è **DNSSEC**, un'estensione del DNS che garantisce all'utente che il sito che sta visitando è quello originale e non una copia creata per truffare.

7. Ripasso

7.1 In sintesi

1. Un protocollo applicativo definisce tipi di messaggi, sintassi, semantica e regole del dialogo.
2. Nel modello client-server i ruoli sono fissi e il server ha IP statico; nel peer-to-peer ogni peer può chiedere e offrire risorse.
3. Telnet dà accesso remoto da terminale su TCP, porta 23, ma tutto viaggia in chiaro: per questo si usa SSH.
4. FTP trasferisce file con due canali: porta 21 per i comandi, porta 20 per i dati; credenziali e file non sono cifrati.

5. In FTP active il canale dati lo apre il server, in FTP passive il client; la passive evita le connessioni in ingresso verso il client.
6. TLS è un livello tra TCP e l'applicazione che cifra la comunicazione: FTP + TLS = FTPS, HTTP + TLS = HTTPS.
7. HTTP è client-server, testuale, pseudoanonimo e stateless; usa la porta 80, HTTPS la 443.
8. HTTP/1.1 introduce connessioni persistenti e cache; HTTP/3 usa UDP al posto di TCP.
9. L'URL è formato da protocollo, host, porta, percorso, query string e fragment; il fragment non arriva al server.
10. I cookie compensano il fatto che HTTP è stateless, permettendo login e sessioni.
11. SMTP invia la posta (porta 25); POP3 la scarica e la toglie dal server, IMAP la lascia sul server e sincronizza più dispositivi.
12. Il DNS traduce i nomi in indirizzi IP con un database distribuito e gerarchico; DNSSEC lo protegge dallo spoofing.

Protocollo	Trasporto	Porta	Porta con TLS
Telnet	TCP	23	–
FTP	TCP	21 comandi, 20 dati	–
HTTP / HTTPS	TCP (HTTP/3: UDP)	80	443
SMTP	TCP	25	465 o 587
POP3	TCP	110	995
IMAP	TCP	143	993

7.2 Domande di autoverifica

Prova a rispondere senza guardare gli appunti, poi controlla nella pagina successiva.

1. Perché in un'architettura client-server il server deve avere un indirizzo IP statico, mentre il client no?
2. Telnet e SSH fanno la stessa cosa: perché oggi si preferisce SSH?
3. Perché FTP usa due porte diverse invece di una sola?
4. Un client si trova in una rete con IP privato dietro un firewall: quale modalità FTP conviene usare, e perché?
5. Che cosa hanno in comune FTPS e HTTPS?
6. Che cosa vuol dire che HTTP è stateless? Quale problema crea e come lo si risolve?
7. Quale miglioramento porta HTTP/1.1 rispetto a HTTP/1.0, e da quale limite di HTTP/1.0 nasce?
8. Nell'URL https://www.sito.it/prodotti/?colore=rosso#recensioni, quali sono il percorso, la query string e il fragment? Quale di queste parti non viene inviata al server?
9. Il browser riceve una risposta con codice 404. Che cosa è successo, e in quale parte del messaggio di risposta si trova il codice?
10. Usi la posta sia dal telefono sia dal PC e vuoi vedere gli stessi messaggi su entrambi: POP3 o IMAP? Perché?
11. Che differenza c'è tra un record A e un record CNAME?
12. Perché il DNS è vulnerabile allo spoofing, e che cosa rischia l'utente?

7.3 Risposte

1. Il server è sempre in attesa di richieste e i client devono sapere sempre dove trovarlo, quindi il suo indirizzo non deve cambiare. Il client invece si collega solo quando gli serve e non deve essere cercato da nessuno.
2. Perché Telnet non cifra la comunicazione: comandi e password viaggiano in chiaro e possono essere intercettati. SSH offre lo stesso accesso remoto, ma cifrato.
3. Per separare i due compiti: la porta 21 serve al canale comandi (autenticazione, richieste del client, risposte del server), la porta 20 al canale dati, dove passano i file.
4. FTP passive. In active è il server che apre la connessione dati verso il client, e una connessione in ingresso verso un IP privato dietro firewall viene bloccata; in passive tutte le connessioni le apre il client.
5. Entrambi aggiungono TLS tra TCP e il protocollo applicativo (FTP o HTTP): il protocollo funziona come prima, ma i dati viaggiano cifrati.
6. Il server non conserva memoria delle richieste precedenti: ogni richiesta è indipendente. Così, per esempio, cambiando pagina il server non sa più che ci siamo autenticati. Si risolve con i cookie, che il browser rimanda al server a ogni richiesta.
7. Le connessioni persistenti. In HTTP/1.0 la connessione si chiude dopo ogni risposta, quindi ogni risorsa di una pagina richiede una nuova connessione; con HTTP/1.1 la stessa connessione serve per più richieste.
8. Percorso: /prodotti/; query string: colore=rosso; fragment: recensioni. Il fragment non viene inviato al server, perché serve solo al browser per posizionarsi nella pagina.
9. La risorsa richiesta non esiste sul server (Not Found). Il codice si trova nella riga di stato, la prima parte della response.
10. IMAP, perché lascia i messaggi sul server e sincronizza tutti i dispositivi. Con POP3 il primo dispositivo che scarica la posta la toglie dal server, e l'altro non la vede più.
11. Il record A associa un nome a un indirizzo IPv4; il record CNAME associa un nome a un altro nome, cioè definisce un alias dello stesso host.
12. Perché non è autenticato ed è lento: un attaccante può rispondere al posto del server vero e associare al nome l'IP di un server malevolo. L'utente scrive l'indirizzo corretto ma finisce su un sito falso. DNSSEC serve a evitarlo.

Cosa è stato corretto rispetto agli appunti

Argomento	Negli appunti	Versione corretta
Telnet, porta	«Usa solo la porta 23»	La 23 è la porta predefinita, ma la sintassi stessa del comando permette di indicarne un'altra.
FTP con TLS	«TLS aggiunto a FTP lo rende sicuro (SFTP)»	FTP con TLS si chiama **FTPS**. SFTP è un protocollo diverso, basato su SSH.
HTTP/2 e HTTPS	«HTTP/2 aggiunge TLS (diventa HTTPS)»	HTTPS è HTTP su TLS ed esisteva già prima di HTTP/2. Con HTTP/2 i browser lo usano solo su connessioni cifrate, ma non è HTTP/2 a «diventare» HTTPS.
Sigla	«TSL»	**TLS**, Transport Layer Security.